Local eCommerce Debug Bot – Step-by-Step Build Guide

Oct 27, 2026 · @Sunil

Before you start

Follow the phases in order; each one ends with a **Verify** step, so don't move on until it passes. Everything runs on your machine, and no code or data leaves your network.

The guide assumes your eCommerce backend is a Node/TypeScript repo called `ecom-backend` using Hapi (an API server plus cron-driven service workers), axios and Postgres. Where your stack differs (NestJS, Prisma, Mongo, fetch), the step says what to swap.

Prerequisites

Item	Minimum	Check command
OS	macOS 13+, Ubuntu 22.04+, or Windows 11 with WSL2	—
RAM	16 GB (32 GB recommended)	—
Disk	40 GB free (models are 5–20 GB each)	`df -h`
Node.js	20 LTS or newer	`node -v`
Docker Desktop / Docker Engine	24+	`docker -v`
git	any recent	`git --version`
VS Code	optional, for the Continue extension	—

If Node is missing, install it with nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh |
bash
# restart the terminal, then:
nvm install 20
nvm use 20
node -v   # should print v20.x
```

Folder layout you will end up with

```
~/work/
  ecom-backend/           # your existing repo (small code changes in Phase
6)
    .knowledge/           # generated + hand-written knowledge (Phases 4–5)
    AGENTS.md             # pointer for any AI tool
  ecom-debug-bot/         # NEW tooling repo you create in Phase 3
    src/extract/          # code extractor
    src/mcp/              # MCP server (the bot's tools)
    src/profile/          # data profiler
    src/search/           # semantic search
    config/environments.yaml
    .env.stage            # secrets, never committed
```

Keeping the bot in a separate repo means your production code only gains the small explain-mode change; everything else lives outside it.

Phase 1 — Install Ollama and the local models

Ollama runs the LLM and the embedding model on your machine and exposes them on `http://localhost:11434`.

1. **Install Ollama**
   - macOS: download from ollama.com and drag to Applications, or `brew install ollama`
   - Linux: `curl -fsSL https://ollama.com/install.sh | sh`
   - Windows: run the installer from ollama.com (or use the Linux command inside WSL2)
2. **Start the server** (the desktop app starts it automatically; on Linux):

   ```
   ollama serve
   ```

3. **Pull a chat model with good tool calling.** Pick by RAM:

```
ollama pull qwen2.5-coder:14b      # 16 GB machines
# ollama pull qwen2.5-coder:32b    # 32 GB+ machines
ollama pull nomic-embed-text       # embeddings for search (Phase 9)
```

4. **Keep it local only.** Ollama binds to 127.0.0.1 by default. Do not set `OLLAMA_HOST=0.0.0.0` unless you need another machine to reach it.
5. **Raise the context window** (default is small; tools return JSON that needs room). Create a file `Modelfile`:

```
FROM qwen2.5-coder:14b
PARAMETER num_ctx 32768
PARAMETER temperature 0.1
```

   Then build a named variant you will use everywhere:

```
ollama create ecom-debugger -f Modelfile
```

Verify

```
ollama list                        # shows ecom-debugger and nomic-embed-text
curl http://localhost:11434/api/chat -d '{
  "model": "ecom-debugger",
  "stream": false,
  "messages": [{"role":"user","content":"What is the weather in Paris?"}],
  "tools": [{"type":"function","function":
{"name":"get_weather","description":"Get weather",
    "parameters":{"type":"object","properties":{"city":
{"type":"string"}},"required":["city"]}}}]
}'
```

The response should contain `tool_calls` with `get_weather` and `{"city":"Paris"}`. That proves the model can call tools, which the bot depends on.

Phase 2 — Install a chat UI (Open WebUI)

Open WebUI gives you a ChatGPT-style page in your browser, talking only to your local Ollama.

1. **Run it in Docker, bound to localhost only, with telemetry off:**

```
docker run -d --name open-webui --restart always \
  -p 127.0.0.1:3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -e ANONYMIZED_TELEMETRY=false \
  -e ENABLE_OPENAI_API=false \
  -v open-webui:/app/backend/data \
  ghcr.io/open-webui/open-webui:main
```

2. Open `http://localhost:3000`, create the first account (it becomes the local admin; nothing is sent anywhere).
3. In the model picker, choose **ecom-debugger**.

Verify: ask "Write a TypeScript function that sums an array." You should get an answer within a few seconds, and `docker logs open-webui` should show no outbound errors.

Prefer working in VS Code? Install the **Continue** extension instead (or as well). In its config, set the model provider to Ollama and the model to `ecom-debugger`. Continue supports MCP servers natively, which makes Phase 7 slightly simpler.

Phase 3 — Create the tooling project

This repo holds the extractor, the bot's MCP server, the profiler and search.

1. **Create and initialise it:**

```
mkdir -p ~/work/ecom-debug-bot && cd ~/work/ecom-debug-bot
git init
npm init -y
npm pkg set type=module
```

2. **Install dependencies:**

```
npm i ts-morph yaml zod dotenv axios fast-glob @modelcontextprotocol/sdk
npm i -D typescript tsx @types/node
```

Package	Why
ts-morph	Reads your TypeScript code as a syntax tree
yaml	Writes compact knowledge files

Package	Why
zod	Validates tool inputs
dotenv	Loads per-environment secrets
axios	Calls downstream APIs
fast-glob	Finds files
@modelcontextprotocol/sdk	Exposes tools to the LLM
tsx	Runs `.ts` files directly

3. **Create `tsconfig.json`:**

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

4. **Create the folders and ignore secrets:**

```
mkdir -p src/extract src/mcp src/profile src/search config
printf "node_modules\ndist\n.env*\n.lancedb\n" > .gitignore
```

5. **Point the tools at your backend** with a `.env` file in the tooling repo:

```
ECOM_REPO=/Users/you/work/ecom-backend
OLLAMA_URL=http://localhost:11434
```

Verify: `npx tsx -e "console.log('ok')"` prints `ok`.

Phase 4 — Generate knowledge from your code

A script reads your backend's TypeScript (no LLM involved) and writes compact facts into `ecom-backend/.knowledge/generated/`. Then a second script asks the local LLM to write a short summary per module.

4.1 Create the extractor

Create `src/extract/index.ts`:

```
import "dotenv/config";
import { Project, SyntaxKind, Node, SourceFile } from "ts-morph";
import { mkdirSync, writeFileSync } from "node:fs";
import { createHash } from "node:crypto";
import path from "node:path";
import YAML from "yaml";

const REPO = process.env.ECOM_REPO!;
const OUT = path.join(REPO, ".knowledge", "generated");
mkdirSync(OUT, { recursive: true });

const project = new Project({ tsConfigFilePath: path.join(REPO,
"tsconfig.json") });
const files = project.getSourceFiles().filter(
  (f) => !/node_modules|\.(spec|test)\.ts$|\.d\.ts$/.test(f.getFilePath())
);

const rel = (p: string) => path.relative(REPO, p);
const HTTP = new Set(["get", "post", "put", "patch", "delete"]);
const NEST = new Set(["Get", "Post", "Put", "Patch", "Delete"]);

const routes: any[] = [];
const outbound: any[] = [];
const envVars: Record<string, Set<string>> = {};
const symbols: Record<string, any[]> = {};
const callGraph: Record<string, string[]> = {};
const manifest: Record<string, string> = {};

// Name of the function/method that contains a node
function owner(node: Node): string {
  for (const a of node.getAncestors()) {
    if (Node.isFunctionDeclaration(a) || Node.isMethodDeclaration(a)) {
      const cls =
```

```
a.getFirstAncestorByKind(SyntaxKind.ClassDeclaration)?.getName();
      return (cls ? cls + "." : "") + (a.getName() ?? "anonymous");
    }
    if (Node.isVariableDeclaration(a)) return a.getName();
  }
  return "<module>";
}

function scanFile(sf: SourceFile) {
  const file = rel(sf.getFilePath());
  manifest[file] =
createHash("sha1").update(sf.getFullText()).digest("hex").slice(0, 12);

  // 1) Express routes + outbound HTTP calls
  for (const call of sf.getDescendantsOfKind(SyntaxKind.CallExpression)) {
    const expr = call.getExpression();
    if (!Node.isPropertyAccessExpression(expr)) continue;
    const method = expr.getName();
    const target = expr.getExpression().getText();
    const [first] = call.getArguments();
    if (!HTTP.has(method)) continue;

    if (/^(app|router|\w+Router)$/.test(target) && first &&
Node.isStringLiteral(first)) {
      routes.push({ method: method.toUpperCase(), path:
first.getLiteralValue(), file,
        handler: call.getArguments().at(-1)?.getText().slice(0, 80) });
    } else if (/axios|http|client|api/i.test(target)) {
      outbound.push({ caller: owner(call), via: target, method:
method.toUpperCase(),
        url: first?.getText().slice(0, 150), file, line:
call.getStartLineNumber() });
    }
  }

  // 2) NestJS routes (@Get('path') on methods)
  for (const m of sf.getDescendantsOfKind(SyntaxKind.MethodDeclaration)) {
    for (const d of m.getDecorators()) {
      if (NEST.has(d.getName())) {
        routes.push({ method: d.getName().toUpperCase(), path:
d.getArguments()[0]?.getText() ?? "''",
          file, handler: owner(m.getBody() ?? m) });
      }
```

```
    }
  }

  // 3) Environment variables (process.env.X)
  for (const pa of
sf.getDescendantsOfKind(SyntaxKind.PropertyAccessExpression)) {
    if (pa.getExpression().getText() === "process.env") {
      (envVars[pa.getName()] ??= new Set()).add(file);
    }
  }

  // 4) Exported symbols with signatures + JSDoc
  for (const [name, decls] of sf.getExportedDeclarations()) {
    for (const d of decls) {
      const entry: any = { name, kind: d.getKindName() };
      if (Node.isFunctionDeclaration(d)) {
        entry.sig = `(${d.getParameters().map((p) => p.getText()).join(",
")}): ${d.getReturnType().getText(d)}`;
        entry.doc = d.getJsDocs().map((j) =>
j.getDescription().trim()).join(" ") || undefined;
      }
      if (Node.isClassDeclaration(d)) entry.methods = d.getMethods().map((m)
=> m.getName());
      if (Node.isInterfaceDeclaration(d) || Node.isTypeAliasDeclaration(d))
entry.def = d.getText().slice(0, 400);
      (symbols[file] ??= []).push(entry);
    }
  }

  // 5) Call graph: function -> names it calls
  const fns = [...sf.getFunctions(), ...sf.getClasses().flatMap((c) =>
c.getMethods())];
  for (const fn of fns) {
    const key = `${file}#${owner(fn.getBody() ?? fn)}`;
    const callees = new Set(
      fn.getDescendantsOfKind(SyntaxKind.CallExpression).map((c) =>
c.getExpression().getText().split("(")[0].slice(0, 60))
    );
    callGraph[key] = [...callees];
  }
}

files.forEach(scanFile);
```

```
const env = Object.fromEntries(Object.entries(envVars).map(([k, v]) => [k,
[...v]]));
writeFileSync(path.join(OUT, "routes.yaml"), YAML.stringify(routes));
writeFileSync(path.join(OUT, "downstream-calls.yaml"),
YAML.stringify(outbound));
writeFileSync(path.join(OUT, "env-vars.yaml"), YAML.stringify(env));
writeFileSync(path.join(OUT, "symbols.yaml"), YAML.stringify(symbols));
writeFileSync(path.join(OUT, "call-graph.json"), JSON.stringify(callGraph));
writeFileSync(path.join(OUT, "manifest.json"), JSON.stringify(manifest, null,
1));
console.log(`files=${files.length} routes=${routes.length}
outbound=${outbound.length} env=${Object.keys(env).length}`);
```

Adjust for your stack: if your HTTP client is a wrapper (for example `this.inventoryClient.get(...)`), the regex `/axios|http|client|api/i` already catches names containing "client" or "api"; add your wrapper's name if it differs. For `fetch(url)`, add a check for `expr.getText() === "fetch"`.

Hapi routes: Hapi declares routes as `server.route({ method, path, handler })` objects, which this extractor doesn't parse. You don't need it to: Phase 7.2 writes the real route table from `server.table()` into `generated/hapi-routes.yaml`. Keep this extractor for outbound calls, env vars, symbols and the call graph.

4.2 Run it

Add scripts to the tooling repo's `package.json`:

```
npm pkg set scripts.extract="tsx src/extract/index.ts"
npm pkg set scripts.summarize="tsx src/extract/summarize.ts"
npm run extract
```

Verify: the console prints non-zero counts, and `ecom-backend/.knowledge/generated/` contains six files. Open `downstream-calls.yaml` and check that your inventory, catalog and offer API calls appear. If a known call is missing, adjust the regex as above.

4.3 Add the module dependency graph

Run inside the backend repo:

```
cd ~/work/ecom-backend
npx madge --ts-config tsconfig.json --extensions ts --json src >
.knowledge/generated/deps.json
npx madge --ts-config tsconfig.json --extensions ts --circular src   # bonus:
lists circular imports
```

4.4 Summarise each module with the local LLM

Create `src/extract/summarize.ts`. It groups files by their folder under `src/` (for example `src/offers`), sends the extracted facts plus trimmed source to Ollama, and writes `.knowledge/modules/<module>.md`:

```
import "dotenv/config";
import { readFileSync, writeFileSync, mkdirSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";

const REPO = process.env.ECOM_REPO!;
const K = path.join(REPO, ".knowledge");
const symbols = YAML.parse(readFileSync(path.join(K,
"generated/symbols.yaml"), "utf8"));
const outbound = YAML.parse(readFileSync(path.join(K, "generated/downstream-
calls.yaml"), "utf8"));
mkdirSync(path.join(K, "modules"), { recursive: true });

// group files by module folder: src/<module>/...
const modules: Record<string, string[]> = {};
for (const file of Object.keys(symbols)) {
  const m = file.split("/")[1] ?? "root";
  (modules[m] ??= []).push(file);
}

const TEMPLATE = `Write a concise module note in Markdown with exactly these
sections:
# <module name>
## Purpose (2 sentences)
## Entry points (bullets: function -> one line)
## Business rules (bullets, each with source file#function)
## Downstream dependencies (bullets: API/DB + why)
## Failure modes (bullets)
Use only facts from the input. Max 350 words.`;
```

```
for (const [mod, files] of Object.entries(modules)) {
  const facts = {
    symbols: Object.fromEntries(files.map((f) => [f, symbols[f]])),
    outbound: outbound.filter((o: any) => files.includes(o.file)),
  };
  const source = files.map((f) => `// ${f}\n` + readFileSync(path.join(REPO,
f), "utf8").slice(0, 6000)).join("\n\n").slice(0, 60000);

  const res = await fetch(`${process.env.OLLAMA_URL}/api/chat`, {
    method: "POST",
    body: JSON.stringify({
      model: "ecom-debugger", stream: false,
      messages: [
        { role: "system", content: TEMPLATE },
        { role: "user", content: `Module:
${mod}\nFACTS:\n${YAML.stringify(facts)}\nSOURCE:\n${source}` },
      ],
    }),
  });
  const { message } = await res.json();
  writeFileSync(path.join(K, "modules", `${mod}.md`), message.content);
  console.log("wrote", mod);
}
```

```
npm run summarize
```

Verify: open `.knowledge/modules/offers.md`. Correct anything wrong by hand; these files are reviewed by you, not trusted blindly. Expect 1–3 minutes per module on a 14B model.

Phase 5 — Write the hand-curated knowledge

Generated files tell an AI *what exists*; these files tell it *how the business works*. Spend real time here, because they are what make answers correct. All go in `ecom-backend/.knowledge/`.

5.1 `INDEX.md` — the only file always loaded (keep under ~60 lines)

```
# Knowledge index (read this first, then load only what you need)

## System in one paragraph
Node/TS backend serving catalog, pricing, offers, inventory and cart for web
+ app.

## Where to look
| Question about | Read |
|---|---|
| Why an offer is/isn't applied | rules/offer-eligibility.yaml,
modules/offers.md |
| Inventory / availability | modules/inventory.md |
| Which API / URL per env | envs/environments.yaml, generated/downstream-
calls.yaml |
| DB tables and fields | data/data-dictionary.yaml |
| What a data job produces | data/job-outputs.yaml |
| Terms (ecomFlag, stackable...) | glossary.yaml |
| HTTP endpoints | generated/routes.yaml |
| Who calls what | generated/call-graph.json (large: search, don't load
whole) |

## Debugging tools available
reproduce_request, run_worker, list_routes, list_workers, call_api, query_db,
read_knowledge, search_code, find_job_traces
```

5.2 `AGENTS.md` at the repo root

Many AI coding tools read this file automatically.

```
# AGENTS.md
Before answering questions about this codebase, read .knowledge/INDEX.md and
load only the files it points to.
Prefer YAML knowledge files over reading source. Open source files only at
the file#function a rule cites.
Never modify .knowledge/generated/* by hand; run `npm run extract` in ecom-
debug-bot.
```

5.3 `rules/offer-eligibility.yaml` — the most important file

Write one entry per rule your offer engine applies, in the order it applies them. Open your offer code and copy each check's real condition.

```
version: 1
entry_point: src/offers/eligibility.ts#evaluateOffer
evaluation_order: [offer_active, ecom_flag, inventory, channel, stackability,
customer_segment]
rules:
  - id: offer_active
    check: offer.status == ACTIVE and now between offer.startDate and
offer.endDate (UTC)
    source: src/offers/rules/offerActive.ts#isActive
    data_from: offers-api GET /offers/{offerId}
    fail_reason: OFFER_INACTIVE
    common_causes: [end date passed in stage, timezone mismatch, offer not
published to stage]
  - id: ecom_flag
    check: product.ecomFlag == 'Y'
    source: src/offers/rules/ecomFlag.ts#isEcomEnabled
    data_from: catalog-api GET /products/{sku}
    fail_reason: OFFER_ECOM_FLAG_OFF
  - id: inventory
    check: availableQty >= config.offers.minQty (default 1) at the requested
store
    source: src/inventory/availability.ts#getAvailable
    data_from: inventory-api GET /inventory/{sku}?storeId={storeId}
    fail_reason: OFFER_NO_INVENTORY
  - id: stackability
    check: no other active offer on the SKU with stackable=false and higher
priority
    source: src/offers/stacking.ts#resolveStack
    data_from: offers-api GET /offers?sku={sku}&status=ACTIVE
    fail_reason: OFFER_NOT_STACKABLE
```

5.4 `envs/environments.yaml` — URLs only, never secrets

```
stage:
  catalog_api: https://catalog.stage.internal.example
  inventory_api: https://inventory.stage.internal.example
  offers_api: https://offers.stage.internal.example
  auth: { type: bearer, token_env: STAGE_API_TOKEN }
  db: { url_env: STAGE_DB_URL_READONLY }
qa:
  catalog_api: https://catalog.qa.internal.example
  inventory_api: https://inventory.qa.internal.example
  offers_api: https://offers.qa.internal.example
  auth: { type: bearer, token_env: QA_API_TOKEN }
  db: { url_env: QA_DB_URL_READONLY }
```

Copy the same file to `ecom-debug-bot/config/environments.yaml` (Phase 6.3 extends it for the runner). The `*_env` fields name variables held in `.env.stage` / `.env.qa`, so tokens never appear in knowledge or prompts.

5.5 `glossary.yaml`

```
ecomFlag: Catalog attribute; 'Y' means the product may be sold online. Set by
merchandising feed.
stackable: Offer attribute; false means it cannot combine with another offer
on the same SKU.
channel: WEB | APP | STORE; offers can target a subset.
```

Verify: paste `INDEX.md` into Open WebUI and ask "Which files would you read to explain why an offer is missing?" It should name the rules file and the offers module.

Phase 6 — Integrate once: the debug runner (Hapi + cron workers)

The runner boots your **real** Hapi server or any cron worker in a child process, configured for one environment (stage, qa…). It records every business function call (arguments + return), every downstream API response and every DB query, blocks all writes, and returns a compact trace. No per-feature code: a new domain is one folder name in a regex.

For the offer question, the bot calls the runner with `GET /products/4455/offers?storeId=12` and reads back a trace like this:

```
GET /products/4455/offers?storeId=12 → 200 []
├ offers/eligibility#getCandidateOffers(["4455"]) → ["OFF-123","OFF-099"]
| ├ HTTP GET https://offers.stage…/offers?sku=4455 → 200 [{id:OFF-
123,stackable:false…}]
├ catalog/product#getProduct(["4455"]) → {ecomFlag:"Y",…}
├ inventory/availability#isAvailable(["4455","12"]) → false
| ├ HTTP GET https://inventory.stage…/inventory/4455?storeId=12 → 200
{availableQty:0}
├ offers/stacking#resolveStack([["OFF-123","OFF-099"]]) → ["OFF-099"]
└ offers/filter#applyEligibility(…) → []
```

The backend needs two small, one-time refactors (6.1, 6.2). Everything else lives in the tooling repo.

6.1 Backend: build the server without starting it

Hapi's `server.inject()` runs a request through all your plugins, auth, validation and handlers without opening a port. To use it, the server must be creatable without `start()`. Many Hapi projects already have this split for `@hapi/lab` tests; if yours does, reuse it.

`src/server.ts`:

```
import Hapi from "@hapi/hapi";
import { registerPlugins } from "./plugins";
import { registerRoutes } from "./routes";

export async function createServer() {
  const server = Hapi.server({ port: Number(process.env.PORT ?? 3000), host:
"0.0.0.0" });
  await registerPlugins(server);
  registerRoutes(server);
  await server.initialize();          // runs onPreStart etc.; ready for
inject(), no port opened
  return server;
}
```

`src/index.ts` (production entry, behaviour unchanged):

```
import { createServer } from "./server";
createServer().then((s) => s.start()).then(() => console.log("server
started"));
```

If a plugin starts background work on initialize (queue consumers, schedulers, cache warmers), skip it in debug mode:

```
if (process.env.DEBUG_RUNNER !== "true") startQueueConsumer();
```

6.2 Backend: a worker registry (separate "what" from "when")

Cron decides *when* a worker runs; the registry exposes *what* it does, so the runner can call any worker on demand, for one SKU, without waiting for the schedule.

`src/workers/registry.ts`:

```
import { syncInventory } from "./inventorySync";
import { publishOffers } from "./offerPublisher";
import { buildProductFeed } from "./productFeed";

export type WorkerArgs = { skus?: string[]; limit?: number; dryRun?: boolean
};

export const WORKERS: Record<string, (args: WorkerArgs) => Promise<unknown>>
= {
  "inventory-sync": syncInventory,
  "offer-publisher": publishOffers,
  "product-feed": buildProductFeed,
};
```

Your cron file keeps the schedule and calls the registry (shown with node-cron; agenda, bull or node-schedule work the same way):

```
import cron from "node-cron";
import { WORKERS } from "./registry";
cron.schedule("*/15 * * * *", () => WORKERS["inventory-sync"]({}));
cron.schedule("0 * * * *", () => WORKERS["offer-publisher"]({}));
```

Give each worker an optional `skus` / `limit` filter, so a debug run processes one product instead of the whole catalog. This is the only change inside worker code.

Build the backend with tsc. The runner loads the compiled `dist/` output (`npm run build`), because tsc-compiled CommonJS exports can be wrapped at load time. Check that `tsconfig.json` has `"module": "commonjs"` and `"sourceMap": true`.

6.3 Describe each environment for the runner

Replace `ecom-debug-bot/config/environments.yaml` with:

```
stage:
  backend_env_file: .env.backend.stage     # env vars your Hapi app reads at
boot
  services:                                # used by the call_api tool
    catalog: https://catalog.stage.internal.example
    inventory: https://inventory.stage.internal.example
    offers: https://offers.stage.internal.example
  auth: { token_env: STAGE_API_TOKEN }
  db_readonly_env: STAGE_DB_URL_READONLY
  allow_post_paths: ["/offers/search"]     # downstream POSTs that only read
qa:
  backend_env_file: .env.backend.qa
  services: { catalog: https://catalog.qa.internal.example }
  auth: { token_env: QA_API_TOKEN }
  db_readonly_env: QA_DB_URL_READONLY
```

`.env.backend.stage` = exactly the variables your deployment sets for stage (`NODE_ENV` , downstream URLs, DB URL, Redis…), with two changes: **the DB user is read-only** and **API tokens are read-only**. If you use the `config` npm package, `NODE_ENV=stage` plus `NODE_CONFIG_DIR=/Users/you/work/ecom-backend/config` is usually enough.

`.env.stage` holds the tooling's own secrets (`STAGE_API_TOKEN` , `STAGE_DB_URL_READONLY`). Both files are git-ignored by the `.env*` rule.

6.4 Install the runner's dependencies (tooling repo)

```
cd ~/work/ecom-debug-bot
npm i require-in-the-middle @opentelemetry/api @opentelemetry/sdk-node \
      @opentelemetry/sdk-trace-base @opentelemetry/auto-instrumentations-node
pg
npm i -D @types/pg
mkdir -p src/runner
```

6.5 Shared config + masking

`src/runner/config.ts` :

```
import { readFileSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";
import dotenv from "dotenv";

export const ROOT = path.resolve(import.meta.dirname, "../..");
export const ALLOWED_ENVS = ["dev", "stage", "qa"] as const;      // no prod
until you trust it
export type Env = (typeof ALLOWED_ENVS)[number];

const all = YAML.parse(readFileSync(path.join(ROOT,
"config/environments.yaml"), "utf8"));
export function cfg(env: Env) {
  if (!ALLOWED_ENVS.includes(env) || !all[env]) throw new Error(`Env ${env}
not allowed or not configured`);
  return all[env];
}
export const secrets = (env: Env) =>
dotenv.parse(readFileSync(path.join(ROOT, `.env.${env}`)));
```

`src/runner/mask.ts`:

```
const PII =
/email|phone|firstName|lastName|address|customerName|card|password|token/i;
export function mask(v: unknown): unknown {
  if (Array.isArray(v)) return v.map(mask);
  if (v && typeof v === "object")
    return Object.fromEntries(Object.entries(v).map(([k, x]) => [k,
PII.test(k) ? "***" : mask(x)]));
  return v;
}
export const short = (v: unknown, n = 400) => {
  try { return JSON.stringify(mask(v))?.slice(0, n) ?? "undefined"; } catch {
return "[unserializable]"; }
};
```

6.6 The instrumentation (the "integrate once" core)

`src/runner/instrument.ts` must load before any backend code. It does four things: starts tracing into memory, wraps business functions, captures and guards axios calls, and guards Postgres writes.

```
import { NodeSDK } from "@opentelemetry/sdk-node";
import { InMemorySpanExporter, SimpleSpanProcessor } from
"@opentelemetry/sdk-trace-base";
import { getNodeAutoInstrumentations } from "@opentelemetry/auto-
instrumentations-node";
import { trace } from "@opentelemetry/api";
import { Hook } from "require-in-the-middle";
import { short } from "./mask.js";

// 1) Tracing into memory (DB, Redis, Hapi spans come from auto-
instrumentation)
export const exporter = new InMemorySpanExporter();
const sdk = new NodeSDK({
  serviceName: "debug-runner",
  spanProcessors: [new SimpleSpanProcessor(exporter)],
  instrumentations: [getNodeAutoInstrumentations({
    "@opentelemetry/instrumentation-fs": { enabled: false },
    "@opentelemetry/instrumentation-http": { enabled: false },   // axios is
captured below, with bodies
  })],
});
sdk.start();
export const shutdown = () => sdk.shutdown();
export const tracer = trace.getTracer("biz");
export const blocked: string[] = [];

// 2) Wrap every exported function in the watched folders of dist/
const BUILD = process.env.BUILD_DIR ?? "dist";
const WATCH = new RegExp(`/${BUILD}/(${process.env.WATCH ??
"offers|pricing|inventory|catalog|cart|promotions|workers|services"})/`);

new Hook(null, { internals: true }, (exports: any, name: string) => {
  const file = name.replace(/\\/g, "/");
  if (!WATCH.test(file) || !exports || typeof exports !== "object") return
exports;
  for (const key of Object.keys(exports)) {
    const fn = exports[key];
    if (typeof fn !== "function" || /^[A-Z]/.test(key)) continue;          //
skip classes
    const label = `${file.split(`/${BUILD}/`)[1].replace(/\.js$/,
"")}#${key}`;
    const wrapped = function (this: unknown, ...args: unknown[]) {
      return tracer.startActiveSpan(label, (span) => {
```

```
        span.setAttribute("biz", true);
        span.setAttribute("args", short(args));
        const done = (r: unknown) => { span.setAttribute("return", short(r));
span.end(); return r; };
        const failed = (e: any) => { span.setAttribute("error",
String(e?.message ?? e)); span.end(); throw e; };
        try {
          const r: any = fn.apply(this, args);
          return r && typeof r.then === "function" ? r.then(done, failed) :
done(r);
        } catch (e) { return failed(e); }
      });
    };
    try { exports[key] = wrapped; } catch { /* non-writable export: skip */ }
  }
  return exports;
});

// 3) axios: record every downstream response; block writes but keep the flow
running
new Hook(["axios"], (axios: any) => {
  const allowPost: string[] = JSON.parse(process.env.ALLOW_POST_PATHS ?? "
[]");
  const attach = (inst: any) => {
    inst.interceptors.request.use((c: any) => {
      const m = String(c.method ?? "get").toUpperCase();
      const url = `${c.baseURL ?? ""}${c.url ?? ""}`;
      if (!["GET", "HEAD"].includes(m) && !allowPost.some((p) =>
url.includes(p))) {
        blocked.push(`HTTP ${m} ${url} ${short(c.data, 300)}`);
        c.adapter = async () => ({ data: { debugRunner: "write blocked" },
status: 200, statusText: "BLOCKED", headers: {}, config: c });
      }
      return c;
    });
    const record = (c: any, status: unknown, body: unknown) =>
      trace.getActiveSpan()?.addEvent("downstream", {
        method: String(c?.method ?? "get").toUpperCase(), url: `${c?.baseURL
?? ""}${c?.url ?? ""}`,
        status: String(status), body: short(body, 600) });
    inst.interceptors.response.use(
      (r: any) => { record(r.config, r.status, r.data); return r; },
      (e: any) => { record(e.config, e.response?.status ?? "ERR",
```

```
e.response?.data ?? e.message); throw e; });
  };
  attach(axios);
  const create = axios.create;
  axios.create = function (...a: any[]) { const inst = create.apply(this, a);
attach(inst); return inst; };
  return axios;
});

// 4) Postgres: let reads through, record and skip writes
new Hook(["pg"], (pg: any) => {
  for (const C of [pg.Client, pg.Pool]) {
    const query = C.prototype.query;
    C.prototype.query = function (text: any, ...rest: any[]) {
      const sql = typeof text === "string" ? text : text?.text ?? "";
      if (!/^\s*(select|show|explain)\b/i.test(sql) && !/^\s*with\b(?!
[\s\S]*\b(insert|update|delete)\b)/i.test(sql)) {
        blocked.push(`SQL ${sql.replace(/\s+/g, " ").slice(0, 300)}`);
        const fake = { rows: [], rowCount: 0, command: "BLOCKED" };
        const cb = rest.find((r) => typeof r === "function");
        if (cb) return void cb(null, fake);
        return Promise.resolve(fake);
      }
      return query.call(this, text, ...rest);
    };
  }
  return pg;
});
```

Notes:

- Calls **between functions in the same file** don't pass through `exports`, so they show as one step. If a critical decision is buried inside a large function, extract it into its own exported function (see 6.9).
- Using `fetch` instead of axios? Re-enable `instrumentation-http` and `instrumentation-undici` (URLs and status, no bodies). Using MongoDB? Add a hook on `mongodb` that wraps `Collection.prototype` write methods (`insertOne`, `updateOne`, `deleteMany` …) the same way as `pg`.
- These guards keep the flow running. The read-only DB user and read-only tokens from 6.3 are the real safety net; keep both.

6.7 The child process: runs one job

`src/runner/child.ts`:

```
import { exporter, shutdown, blocked, tracer } from "./instrument.js";   //
MUST be the first import
import { createRequire } from "node:module";
import { writeFileSync } from "node:fs";
import path from "node:path";
import { compress } from "./compress.js";
import { short } from "./mask.js";

const job = JSON.parse(process.argv[2]);
const req = createRequire(import.meta.url);
const load = (p: string) => req(path.join(process.cwd(),
process.env.BUILD_DIR ?? "dist", p));

async function run(): Promise<unknown> {
  switch (job.kind) {
    case "request": {
      const server = await load("server.js").createServer();
      const res = await server.inject({ method: job.method ?? "GET", url:
job.url, headers: job.headers, payload: job.payload });
      await server.stop();
      return { status: res.statusCode, body: res.result ?? res.payload };
    }
    case "worker": {
      const { WORKERS } = load("workers/registry.js");
      if (!WORKERS[job.name]) throw new Error(`Unknown worker ${job.name}.
Known: ${Object.keys(WORKERS).join(", ")}`);
      return WORKERS[job.name]({ ...job.args, dryRun: true });
    }
    case "routes": {
      const server = await load("server.js").createServer();
      const routes = server.table().map((r: any) => ({ method:
r.method.toUpperCase(), path: r.path,
        description: r.settings.description, tags: r.settings.tags }));
      await server.stop();
      return routes;
    }
    case "workers":
      return Object.keys(load("workers/registry.js").WORKERS);
  }
```

```
  throw new Error(`Unknown job kind ${job.kind}`);
}

let result: unknown, error: string | undefined;
try {
  result = await tracer.startActiveSpan("job", async (span) => { try { return
await run(); } finally { span.end(); } });
} catch (e: any) { error = String(e?.stack ?? e).slice(0, 2000); }
await new Promise((r) => setTimeout(r, 300));        // let late async spans
finish

const listing = job.kind === "routes" || job.kind === "workers";
writeFileSync(job.out, JSON.stringify({
  result: listing ? result : short(result, 3000),
  error,
  trace: listing ? undefined : compress(exporter.getFinishedSpans()),
  blocked_writes: blocked,
}));
await shutdown();
process.exit(0);
```

The result goes to a file, not stdout, because your Hapi app's own logs also print to stdout.

6.8 Trace compression and the parent launcher

`src/runner/compress.ts` turns hundreds of spans into a short tree (typically 1–3k tokens):

```
import type { ReadableSpan } from "@opentelemetry/sdk-trace-base";

const sid = (s: ReadableSpan) => s.spanContext().spanId;
const pid = (s: any): string | undefined => s.parentSpanContext?.spanId ??
s.parentSpanId;
const t = (s: ReadableSpan) => s.startTime[0] * 1e9 + s.startTime[1];

function label(s: any): string | null {
  const a = s.attributes;
  if (a.biz) return `${s.name}(${a.args}) → ${a.error ? "THROW " + a.error :
a.return}`;
  const stmt = a["db.query.text"] ?? a["db.statement"];
  if (stmt) return `DB ${String(stmt).replace(/\s+/g, " ").slice(0, 200)}`;
  return null;                                    // Hapi internals, Redis
pings, etc. are folded away
```

```
}

export function compress(spans: ReadableSpan[], maxLines = 150): string {
  const ids = new Set(spans.map(sid));
  const kids = new Map<string, ReadableSpan[]>();
  for (const s of spans) { const p = pid(s); if (p && ids.has(p)) kids.set(p,
[...(kids.get(p) ?? []), s]); }
  const lines: string[] = [];
  const pad = (d: number) => "| ".repeat(d) + "├ ";

  const walk = (s: any, depth: number) => {
    const l = label(s);
    const d = l ? depth + 1 : depth;
    if (l) lines.push(pad(depth) + l);
    for (const e of s.events ?? []) {
      if (e.name !== "downstream") continue;
      const x = e.attributes;
      lines.push(pad(d) + `HTTP ${x.method} ${x.url} → ${x.status}
${x.body}`);
    }
    const children = (kids.get(sid(s)) ?? []).sort((a, b) => t(a) - t(b));
    for (let i = 0; i < children.length; ) {
      let j = i;
      while (j + 1 < children.length && children[j + 1].name ===
children[i].name) j++;
      if (j - i >= 3) { walk(children[i], d); lines.push(pad(d) + `…
${children[i].name} ×${j - i} more`); }
      else for (let k = i; k <= j; k++) walk(children[k], d);
      i = j + 1;
    }
  };
  spans.filter((s) => !pid(s) || !ids.has(pid(s)!)).sort((a, b) => t(a) -
t(b)).forEach((r) => walk(r, 0));
  return lines.length > maxLines
    ? lines.slice(0, maxLines).join("\n") + `\n… ${lines.length - maxLines}
more lines truncated`
    : lines.join("\n");
}
```

`src/runner/runner.ts` starts one child per job, with the backend repo as working directory and the chosen environment's variables:

```
import { spawn } from "node:child_process";
import { mkdtempSync, readFileSync } from "node:fs";
import os from "node:os";
import path from "node:path";
import dotenv from "dotenv";
import { ROOT, cfg, type Env } from "./config.js";

export async function runJob(env: Env, job: Record<string, unknown>,
timeoutMs = 120_000) {
  const c = cfg(env);
  const backendEnv = dotenv.parse(readFileSync(path.join(ROOT,
c.backend_env_file)));
  const out = path.join(mkdtempSync(path.join(os.tmpdir(), "dbg-")),
"out.json");
  const child = spawn(path.join(ROOT, "node_modules/.bin/tsx"),
[path.join(ROOT, "src/runner/child.ts"), JSON.stringify({ ...job, out })], {
    cwd: process.env.ECOM_REPO,
    env: { ...process.env, ...backendEnv, DEBUG_RUNNER: "true",
ALLOW_POST_PATHS: JSON.stringify(c.allow_post_paths ?? []) },
    stdio: ["ignore", "ignore", "pipe"],
  });
  let stderr = "";
  child.stderr!.on("data", (d) => { stderr = (stderr + d).slice(-4000); });
  await new Promise((resolve, reject) => {
    const timer = setTimeout(() => { child.kill("SIGKILL"); reject(new
Error("runner timed out")); }, timeoutMs);
    child.on("exit", (code) => { clearTimeout(timer); code === 0 ?
resolve(null) : reject(new Error(`runner exit ${code}:
${stderr.slice(-1500)}`)); });
  });
  return JSON.parse(readFileSync(out, "utf8"));
}
```

And a small CLI for testing, `src/runner/cli.ts`:

```
import "dotenv/config";
import { runJob } from "./runner.js";
const [env, kind, a, b] = process.argv.slice(2);
const job = kind === "request" ? { kind, method: "GET", url: a }
          : kind === "worker" ? { kind, name: a, args: JSON.parse(b ?? "{}")
}
          : { kind };
const r = await runJob(env as any, job);
console.log(r.trace ?? ""); console.log("RESULT:", r.result); if (r.error)
console.log("ERROR:", r.error);
if (r.blocked_writes?.length) console.log("BLOCKED WRITES:\n" +
r.blocked_writes.join("\n"));
```

6.9 Verify

```
cd ~/work/ecom-backend && npm run build
cd ~/work/ecom-debug-bot
npm pkg set scripts.run-debug="tsx src/runner/cli.ts"
npm run run-debug -- stage routes                                    # every
Hapi route, from server.table()
npm run run-debug -- stage workers                                   # every
registered worker
npm run run-debug -- stage request "/products/4455/offers?storeId=12"
npm run run-debug -- stage worker offer-publisher '{"skus":["4455"]}'
```

You should see a trace tree like the one at the top of this phase, and for the worker a `BLOCKED WRITES` list showing exactly what it *would* have written. If the tree has few business lines, widen the `WATCH` folder list (environment variable `WATCH="offers|pricing|…"`) to match your `src/` layout.

When the trace isn't enough: if one big function makes the whole decision internally, you'll only see its input and output. Fix those gradually: extract the decision into a small exported function that returns a reason (for example `{ eligible: false, reason: "OFFER_NOT_STACKABLE", blockedBy: "OFF-099" }`). The runner picks it up automatically.

Phase 7 — The gateway: one MCP server, generic tools

MCP (Model Context Protocol) is a standard way to give an LLM tools. The gateway is the single place the bot connects to. Its tools are generic, so they work for every route, worker and service without new code.

Tool	What it does	Typical question
`reproduce_request`	Runs a real Hapi request in an env through the runner, returns trace + response	"Why is offer X missing for SKU Y in stage?"
`run_worker`	Runs a cron worker for given SKUs (dry run), returns trace + blocked writes	"Why didn't the feed job publish SKU Y?"
`list_routes` / `list_workers`	Lists what exists (from `server.table()` and the registry)	"Which endpoint returns offers?"
`call_api`	Direct GET to a downstream service	"What does inventory say right now?"
`query_db`	Read-only SELECT on the env's replica	"What's stored for SKU Y?"
`read_knowledge`	Reads a `.knowledge/` file	Business rules, glossary, data meanings
`search_code` (Phase 9), `find_job_traces` (Phase 8.5)	Search code; past job traces	"Where do we handle BOGO?"

7.1 Create the server

`src/mcp/server.ts`:

```
import "dotenv/config";
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";
import { readFileSync } from "node:fs";
```

```
import path from "node:path";
import axios from "axios";
import pg from "pg";
import { runJob } from "../runner/runner.js";
import { cfg, secrets, ALLOWED_ENVS } from "../runner/config.js";
import { mask } from "../runner/mask.js";

const KNOWLEDGE = path.join(process.env.ECOM_REPO!, ".knowledge");
const env = z.enum(ALLOWED_ENVS);
const out = (data: unknown) => ({ content: [{ type: "text" as const,
  text: (typeof data === "string" ? data :
JSON.stringify(mask(data))).slice(0, 12_000) }] });
const fail = (e: any) => out({ error: e?.response?.status ? `HTTP
${e.response.status}` : String(e?.message ?? e) });
const traced = (r: any) => out([
  r.error ? `ERROR: ${r.error}` : `RESULT: ${r.result}`,
  `TRACE:\n${r.trace}`,
  r.blocked_writes?.length ? `BLOCKED WRITES (would have
written):\n${r.blocked_writes.join("\n")}` : "",
].join("\n\n"));

const server = new McpServer({ name: "ecom-debug", version: "0.2.0" });

server.registerTool("reproduce_request", {
  description: "Run a real request through our Hapi server in the given env
(read-only) and return the response plus a trace of every business function,
downstream API call and DB query. Use first for any 'why does endpoint X
return Y' question.",
  inputSchema: { env, url: z.string().describe("path + query, e.g.
/products/4455/offers?storeId=12"),
    method: z.enum(["GET", "POST", "PUT", "DELETE"]).default("GET"),
    headers: z.record(z.string()).optional(), payload: z.any().optional() },
}, async ({ env, url, method, headers, payload }) => {
  try { return traced(await runJob(env, { kind: "request", url, method,
headers, payload })); } catch (e) { return fail(e); }
});

server.registerTool("run_worker", {
  description: "Run one cron worker in dry-run mode for specific SKUs in the
given env. Returns its trace and the writes it would have made.",
  inputSchema: { env, name: z.string(), skus: z.array(z.string()).optional(),
limit: z.number().int().max(100).optional() },
}, async ({ env, name, skus, limit }) => {
```

```
  try { return traced(await runJob(env, { kind: "worker", name, args: { skus,
limit } }, 300_000)); } catch (e) { return fail(e); }
});

server.registerTool("list_routes", {
  description: "List all Hapi routes (method, path, description).",
inputSchema: { env },
}, async ({ env }) => { try { return out((await runJob(env, { kind: "routes"
})).result); } catch (e) { return fail(e); } });

server.registerTool("list_workers", {
  description: "List all registered cron workers.", inputSchema: { env },
}, async ({ env }) => { try { return out((await runJob(env, { kind: "workers"
})).result); } catch (e) { return fail(e); } });

server.registerTool("call_api", {
  description: "GET a downstream service directly, e.g. service='inventory',
path='/inventory/4455', query={storeId:'12'}.",
  inputSchema: { env, service: z.string(), path: z.string(), query:
z.record(z.string()).optional() },
}, async ({ env, service, path: p, query }) => {
  const c = cfg(env);
  const base = c.services?.[service];
  if (!base) return out({ error: `Unknown service. Known:
${Object.keys(c.services ?? {}).join(", ")}` });
  const token = secrets(env)[c.auth?.token_env];
  try {
    const r = await axios.get(base + p, { params: query, timeout: 10_000,
headers: token ? { Authorization: `Bearer ${token}` } : {} });
    return out({ status: r.status, data: r.data });
  } catch (e) { return fail(e); }
});

server.registerTool("query_db", {
  description: "Run ONE read-only SQL SELECT on the env's read replica.
Returns at most 200 rows. Check .knowledge/data/data-dictionary.yaml for
table names first.",
  inputSchema: { env, sql: z.string() },
}, async ({ env, sql }) => {
  const clean = sql.trim().replace(/;\s*$/, "");
  if (!/^(select|with)\b/i.test(clean) || clean.includes(";") ||
/\b(insert|update|delete|drop|alter|truncate|grant|create|copy)\b/i.test(clea
n))
```

```
    return out({ error: "Only a single SELECT statement is allowed" });
  const client = new pg.Client({ connectionString: secrets(env)
[cfg(env).db_readonly_env] });
  try {
    await client.connect();
    await client.query("SET statement_timeout = 10000; SET
default_transaction_read_only = on;");
    const r = await client.query(`SELECT * FROM (${clean}) q LIMIT 200`);
    return out({ rowCount: r.rowCount, rows: r.rows });
  } catch (e) { return fail(e); } finally { await client.end().catch(() =>
{}); }
});

server.registerTool("read_knowledge", {
  description: "Read a file from the .knowledge folder, e.g. 'INDEX.md' or
'rules/offer-eligibility.yaml'.",
  inputSchema: { file: z.string() },
}, async ({ file }) => {
  const p = path.resolve(KNOWLEDGE, file);
  if (!p.startsWith(KNOWLEDGE)) return out({ error: "Path outside .knowledge"
});
  try { return out(readFileSync(p, "utf8")); } catch (e) { return fail(e); }
});

await server.connect(new StdioServerTransport());
console.error("ecom-debug gateway running");    // stderr only: stdout is the
MCP channel
```

In an MCP server over stdio, **stdout is the protocol channel**. Log with `console.error` only. The backend's own stdout logs are safe because they run in the runner's child process, whose stdout is discarded.

7.2 Save routes and workers as knowledge

The real route table is better than anything parsed from source. Add `src/runner/catalog.ts`:

```
import "dotenv/config";
import { writeFileSync, mkdirSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";
import { runJob } from "./runner.js";

const dir = path.join(process.env.ECOM_REPO!, ".knowledge/generated");
mkdirSync(dir, { recursive: true });
writeFileSync(path.join(dir, "hapi-routes.yaml"), YAML.stringify((await
runJob("stage", { kind: "routes" })).result));
writeFileSync(path.join(dir, "workers.yaml"), YAML.stringify((await
runJob("stage", { kind: "workers" })).result));
console.log("route + worker catalog written");
```

```
npm pkg set scripts.catalog="tsx src/runner/catalog.ts"
npm run catalog
```

Add the cron schedule of each worker to `.knowledge/generated/workers.yaml` by hand once (or extract it from your cron file), so the bot knows when each one runs.

7.3 Test the tools without any LLM

```
npm pkg set scripts.mcp="tsx src/mcp/server.ts"
npx @modelcontextprotocol/inspector npx tsx src/mcp/server.ts
```

The Inspector opens in your browser. Click **List Tools**, choose `reproduce_request`, enter `stage` and `/products/4455/offers?storeId=12`, and run it. Then try `run_worker` with a worker name and one SKU.

Verify: you get `RESULT`, a `TRACE` tree and, for workers, `BLOCKED WRITES`. Fix boot errors (missing env vars in `.env.backend.stage`, DB/Redis connection) here before adding the LLM. A stage reproduction usually takes 3–10 seconds, mostly server boot.

7.4 Connect the gateway to your chat UI

Option A — Continue (VS Code). Add to `~/.continue/config.yaml`:

```
models:
  - name: ecom-debugger
    provider: ollama
    model: ecom-debugger
    roles: [chat]
    capabilities: [tool_use]
mcpServers:
  - name: ecom-debug
    command: npx
    args: ["tsx", "/Users/you/work/ecom-debug-bot/src/mcp/server.ts"]
    env:
      ECOM_REPO: /Users/you/work/ecom-backend
```

Switch Continue to **Agent** mode and the tools appear.

Option B — Open WebUI. Expose the gateway over HTTP with the `mcpo` proxy, bound to localhost:

```
curl -LsSf https://astral.sh/uv/install.sh | sh     # installs uv / uvx (one
time)
cd ~/work/ecom-debug-bot
uvx mcpo --host 127.0.0.1 --port 8000 -- npx tsx src/mcp/server.ts
```

In Open WebUI: **Settings → Tools → add** `http://localhost:8000`, then enable the ecom-debug tools in a chat with the tools toggle beside the message box.

7.5 Give the bot its instructions

In Open WebUI create a Workspace → Model based on `ecom-debugger` (in Continue, add it as a rule) with this system prompt:

```
You debug our eCommerce backend (Hapi API + cron workers). Environments: dev,
stage, qa.
1. Start by calling read_knowledge("INDEX.md"); load only the files it points
to.
2. If you don't know the endpoint or worker, call list_routes or
list_workers.
3. To explain behaviour, REPRODUCE it: reproduce_request for API questions,
run_worker for job questions.
   Ask for env, SKU, store or offer id if missing.
4. Read the TRACE top to bottom. Find the step where the expected item
disappears or the value changes.
5. Confirm suspicious data with call_api or query_db. Base every claim on
tool output; quote values.
6. Never guess data. If a tool errors, say so and stop.
Answer format: Verdict (1 line) -> Where it happened (function + evidence) ->
All contributing causes -> Fix / next step.
```

Verify — your target scenario: ask "Offer OFF-123 is not showing for SKU 4455 in stage, store 12. Why?" The bot should read INDEX.md, find the offers route, call `reproduce_request`, and point to the exact functions where OFF-123 dropped out (no inventory at store 12, OFF-099 won stacking), with the values from the trace. Follow up with "Is the offer-publisher job the cause?" and it should call `run_worker` for that SKU.

Phase 8 — Observe your data jobs

Three pieces: **traces** show what each job run did step by step, **reason-coded logs** explain why each record was classified or skipped, and a **profiler** records what the output data looks like.

8.1 Run Jaeger locally (trace viewer)

```
docker run -d --name jaeger --restart always \
  -e COLLECTOR_OTLP_ENABLED=true \
  -p 127.0.0.1:16686:16686 -p 127.0.0.1:4318:4318 \
  jaegertracing/all-in-one:latest
```

UI: `http://localhost:16686`. Port 4318 receives traces.

8.2 Add OpenTelemetry to the backend's jobs

```
cd ~/work/ecom-backend
npm i @opentelemetry/api @opentelemetry/sdk-node @opentelemetry/auto-
instrumentations-node \
      @opentelemetry/exporter-trace-otlp-http pino
```

Create `src/observability/tracing.ts`:

```
import { NodeSDK } from "@opentelemetry/sdk-node";
import { OTLPTraceExporter } from "@opentelemetry/exporter-trace-otlp-http";
import { getNodeAutoInstrumentations } from "@opentelemetry/auto-
instrumentations-node";

if (process.env.OTEL_ENABLED === "true") {
  const sdk = new NodeSDK({
    serviceName: process.env.OTEL_SERVICE_NAME ?? "ecom-jobs",
    traceExporter: new OTLPTraceExporter({ url:
"http://localhost:4318/v1/traces" }),
    instrumentations: [getNodeAutoInstrumentations()],   // auto-traces HTTP,
pg, mongodb, redis...
  });
  sdk.start();
  process.on("SIGTERM", () => sdk.shutdown());
}
```

Make it the **first import** in each job's entry file: `import "../observability/tracing";`

8.3 Add spans and reason codes inside a job

Create `src/observability/logger.ts`:

```
import pino from "pino";
export const logger = pino({ level: process.env.LOG_LEVEL ?? "info" });
```

Then wrap each stage and each classification decision. Example for a product-feed job:

```
import { trace } from "@opentelemetry/api";
import { randomUUID } from "node:crypto";
import { logger } from "../observability/logger";
const tracer = trace.getTracer("product-feed-job");

export async function runProductFeedJob() {
```

```
  const runId = randomUUID();
  const log = logger.child({ job: "product-feed", runId });
  await tracer.startActiveSpan("product-feed.run", async (run) => {
    run.setAttribute("job.runId", runId);
    const rows = await tracer.startActiveSpan("extract", async (s) => { const
r = await loadFeed(); s.setAttribute("records.in", r.length); s.end(); return
r; });

    for (const row of rows) {
      await tracer.startActiveSpan("classify", async (s) => {
        const result = classify(row);                       // your existing
logic
        s.setAttributes({ sku: row.sku, "classification.result":
result.category, "classification.reason": result.reason });
        log.info({ sku: row.sku, category: result.category, reason:
result.reason }, "classified");
        if (result.category === "SKIPPED") log.warn({ sku: row.sku, reason:
result.reason }, "skipped");
        s.end();
      });
    }
    run.end();
  });
}
```

Make `classify` return a `reason` string such as `SKIPPED_NO_PRICE` or `CLASSIFIED_AS_BUNDLE`, the same pattern as the offer rules. Write these codes into `.knowledge/data/job-outputs.yaml` (below) with a one-line meaning each.

Run the job with tracing and logs to a file:

```
OTEL_ENABLED=true npx tsx src/jobs/productFeed.ts | tee logs/product-
feed.jsonl
```

Verify: in Jaeger, choose service `ecom-jobs` → Find Traces. You should see one `product-feed.run` trace with `extract`, `classify` and auto-traced HTTP/DB child spans. Search by tag `sku=4455` to follow one product through the job.

Per-record spans can be large. For jobs over ~50k records, only create `classify` spans for skipped/unusual records and rely on logs for the rest.

8.4 Profile job outputs with DuckDB

In the tooling repo:

```
cd ~/work/ecom-debug-bot
npm i @duckdb/node-api
npm pkg set scripts.profile="tsx src/profile/index.ts"
```

Describe each job's outputs in `config/jobs.yaml`:

```
- job: product-feed
  outputs:
    - name: classified_products
      kind: file
      source: /data/feeds/out/classified/*.json       # json, csv or parquet
    - name: product_offers
      kind: postgres
      source: public.product_offers
      db_env: STAGE_DB_URL_READONLY
```

Create `src/profile/index.ts`:

```
import "dotenv/config";
import { DuckDBInstance } from "@duckdb/node-api";
import { readFileSync, writeFileSync, mkdirSync } from "node:fs";
import path from "node:path";
import dotenv from "dotenv";
import YAML from "yaml";

const jobs = YAML.parse(readFileSync("config/jobs.yaml", "utf8"));
const secrets = dotenv.parse(readFileSync(".env.stage"));
const db = await DuckDBInstance.create(":memory:");
const conn = await db.connect();
const q = async (sql: string) => (await
conn.runAndReadAll(sql)).getRowObjectsJson();

function fromClause(o: any) {
  if (o.kind === "file") {
    const ext = path.extname(o.source);
    const fn = ext === ".csv" ? "read_csv_auto" : ext === ".parquet" ?
"read_parquet" : "read_json_auto";
    return `${fn}('${o.source}')`;
  }
```

```
  return `pg.${o.source}`;
}

const report: any[] = [];
for (const job of jobs) {
  for (const o of job.outputs) {
    if (o.kind === "postgres") {
      await q("INSTALL postgres; LOAD postgres;");
      await q("DETACH DATABASE IF EXISTS pg;");
      await q(`ATTACH '${secrets[o.db_env]}' AS pg (TYPE postgres,
READ_ONLY);`);
    }
    const src = `(SELECT * FROM ${fromClause(o)} USING SAMPLE 20000 ROWS)`;
    const summary = await q(`SUMMARIZE SELECT * FROM ${src}`);
    const columns: any[] = [];
    for (const c of summary as any[]) {
      const col: any = { name: c.column_name, type: c.column_type, null_pct:
c.null_percentage, distinct_approx: c.approx_unique, min: c.min, max: c.max
};
      if (Number(c.approx_unique) <= 20) {     // looks like a category/enum
-> show its values
        col.top_values = await q(`SELECT "${c.column_name}" AS v, count(*) AS
n FROM ${src} GROUP BY 1 ORDER BY 2 DESC LIMIT 10`);
      }
      columns.push(col);
    }
    report.push({ job: job.job, output: o.name, source: o.source,
profiled_at: new Date().toISOString(), columns });
    console.log("profiled", job.job, o.name);
  }
}
const outDir = path.join(process.env.ECOM_REPO!, ".knowledge/data");
mkdirSync(outDir, { recursive: true });
writeFileSync(path.join(outDir, "job-outputs.generated.yaml"),
YAML.stringify(report));
```

```
npm run profile
```

`INSTALL postgres` downloads a DuckDB extension once. On a fully offline machine, run it once while connected, or copy the extension file from another machine.

Verify: `.knowledge/data/job-outputs.generated.yaml` lists every column with type, null %, distinct count, and the value distribution of category-like columns (for example how many products landed in each classification). Keep a hand-written `job-outputs.yaml` next to it with the meaning of each reason code and category.

8.5 Let the bot read traces

Add one more tool to `src/mcp/server.ts`:

```
server.registerTool("find_job_traces", {
  description: "Find recent job traces in Jaeger for a SKU, to see how a data
job processed/classified it.",
  inputSchema: { sku: z.string(), service: z.string().default("ecom-jobs") },
}, async ({ sku, service }) => {
  const tags = encodeURIComponent(JSON.stringify({ sku }));
  const res = await fetch(`http://localhost:16686/api/traces?
service=${service}&tags=${tags}&limit=5`);
  const { data } = await res.json();
  const spans = data.flatMap((t: any) => t.spans.map((s: any) => ({
    trace: t.traceID, op: s.operationName, start: new Date(s.startTime /
1000).toISOString(),
    tags: Object.fromEntries(s.tags.map((x: any) => [x.key, x.value])) })));
  return out(spans.slice(0, 50));
});
```

8.6 Optional: Metabase for visual exploration

```
docker run -d --name metabase -p 127.0.0.1:3001:3000 -e
MB_ANON_TRACKING_ENABLED=false metabase/metabase
```

Open `http://localhost:3001`, connect the read-only stage database, and build quick charts of classification counts per run. This is for you to understand the logic before documenting it.

Phase 9 — Semantic code search (LanceDB)

This answers fuzzy questions such as "where do we handle BOGO?" by searching meaning, not exact words. LanceDB is an embedded database: a folder on disk, no server to run.

9.1 Install

```
cd ~/work/ecom-debug-bot
npm i @lancedb/lancedb
npm pkg set scripts.index="tsx src/search/index.ts"
```

9.2 Shared embedding helper

Create `src/search/embed.ts`:

```
// nomic-embed-text works best with these task prefixes
export async function embed(texts: string[], kind: "doc" | "query"):
Promise<number[][]> {
  const prefix = kind === "doc" ? "search_document: " : "search_query: ";
  const res = await fetch(`${process.env.OLLAMA_URL ??
"http://localhost:11434"}/api/embed`, {
    method: "POST",
    body: JSON.stringify({ model: "nomic-embed-text", input: texts.map((t) =>
prefix + t) }),
  });
  return (await res.json()).embeddings;
}
```

9.3 Build the index (one chunk per function, one per knowledge section)

Create `src/search/index.ts`:

```
import "dotenv/config";
import * as lancedb from "@lancedb/lancedb";
import { Project } from "ts-morph";
import fg from "fast-glob";
import { readFileSync } from "node:fs";
import path from "node:path";
import { embed } from "./embed.js";

const REPO = process.env.ECOM_REPO!;
const chunks: { id: string; file: string; symbol: string; text: string }[] =
[];

// Code: one chunk per function / method
const project = new Project({ tsConfigFilePath: path.join(REPO,
"tsconfig.json") });
```

```
for (const sf of project.getSourceFiles()) {
  const file = path.relative(REPO, sf.getFilePath());
  if (/node_modules|\.d\.ts$|\.(spec|test)\.ts$/.test(file)) continue;
  const fns = [...sf.getFunctions(), ...sf.getClasses().flatMap((c) =>
c.getMethods())];
  for (const fn of fns) {
    const symbol = fn.getName() ?? "anonymous";
    chunks.push({ id: `${file}#${symbol}`, file, symbol, text:
fn.getText().slice(0, 4000) });
  }
}

// Knowledge: one chunk per markdown section / yaml file
for (const f of await fg(["**/*.md", "**/*.yaml"], { cwd: path.join(REPO,
".knowledge"), ignore: ["generated/**"] })) {
  const text = readFileSync(path.join(REPO, ".knowledge", f), "utf8");
  text.split(/\n(?=## )/).forEach((part, i) =>
    chunks.push({ id: `.knowledge/${f}#${i}`, file: `.knowledge/${f}`,
symbol: part.split("\n")[0].slice(0, 80), text: part.slice(0, 4000) }));
}

// Embed in batches of 32
const rows: any[] = [];
for (let i = 0; i < chunks.length; i += 32) {
  const batch = chunks.slice(i, i + 32);
  const vectors = await embed(batch.map((c) => `${c.file}
${c.symbol}\n${c.text}`), "doc");
  batch.forEach((c, j) => rows.push({ ...c, vector: vectors[j] }));
  console.error(`embedded ${rows.length}/${chunks.length}`);
}

const db = await lancedb.connect(path.resolve(".lancedb"));
await db.createTable("chunks", rows, { mode: "overwrite" });
console.error("index built");
```

```
npm run index
```

Expect a few minutes for a mid-size repo (thousands of functions).

9.4 Add the search tool to the bot

In `src/mcp/server.ts` add:

```
import * as lancedb from "@lancedb/lancedb";
import { embed } from "../search/embed.js";

server.registerTool("search_code", {
  description: "Semantic search over source functions and knowledge notes.
Returns file#function and a snippet.",
  inputSchema: { query: z.string(), limit:
z.number().int().max(10).default(5) },
}, async ({ query, limit }) => {
  const [vector] = await embed([query], "query");
  const table = await (await
lancedb.connect(path.resolve(import.meta.dirname,
"../../.lancedb"))).openTable("chunks");
  const hits = await table.search(vector).limit(limit).toArray();
  return out(hits.map((h: any) => ({ id: h.id, snippet: h.text.slice(0, 600)
})));
});
```

Verify: in the Inspector (Phase 7.3) run `search_code` with "buy one get one free". The top hits should be your BOGO/promotion functions. Snippets are capped at 600 characters to keep token use low; the bot can open the full function with `read_knowledge` or ask you.

Phase 10 — Keep the knowledge fresh automatically

Fast steps run on every commit; slow LLM steps run nightly and only for modules whose code changed.

What	When	Time
`extract` (facts) + `check-rules`	every commit (pre-commit hook)	seconds
`summarize` (changed modules only) + `index` + `profile`	nightly, 02:00	minutes
Review hand-written rules	when `check-rules` fails or offer code changes	—

10.1 Detect which modules changed

Create `src/extract/changed.ts` in the tooling repo:

```
import "dotenv/config";
import { readFileSync, existsSync, copyFileSync } from "node:fs";
import path from "node:path";

const G = path.join(process.env.ECOM_REPO!, ".knowledge/generated");
const prevPath = path.join(G, "manifest.prev.json");
const prev = existsSync(prevPath) ? JSON.parse(readFileSync(prevPath,
"utf8")) : {};
const curr = JSON.parse(readFileSync(path.join(G, "manifest.json"), "utf8"));
const changed = new Set(Object.keys(curr).filter((f) => prev[f] !==
curr[f]).map((f) => f.split("/")[1]));
copyFileSync(path.join(G, "manifest.json"), prevPath);
process.stdout.write([...changed].join(","));
```

Then make `summarize.ts` skip unchanged modules. Add near the top of its loop:

```
const only = process.env.MODULES?.split(",").filter(Boolean);
// inside: for (const [mod, files] of Object.entries(modules)) {
if (only && !only.includes(mod)) continue;
```

10.2 Check that hand-written rules still point at real code

Create `src/extract/check-rules.ts`:

```
import "dotenv/config";
import { readFileSync, existsSync } from "node:fs";
import path from "node:path";
import YAML from "yaml";

const REPO = process.env.ECOM_REPO!;
const rules = YAML.parse(readFileSync(path.join(REPO,
".knowledge/rules/offer-eligibility.yaml"), "utf8")).rules;
let bad = 0;
for (const r of rules) {
  const [file, fn] = r.source.split("#");
  const p = path.join(REPO, file);
  if (!existsSync(p) || (fn && !readFileSync(p, "utf8").includes(fn))) {
    console.error(`STALE RULE ${r.id}: ${r.source} not found`); bad++;
  }
}
process.exit(bad ? 1 : 0);
```

Add the scripts:

```
cd ~/work/ecom-debug-bot
npm pkg set scripts.check-rules="tsx src/extract/check-rules.ts"
npm pkg set scripts.nightly='MODULES=$(tsx src/extract/changed.ts) npm run
summarize && npm run index && npm run profile'
```

10.3 Pre-commit hook in the backend (Husky)

```
cd ~/work/ecom-backend
npm i -D husky
npx husky init
cat > .husky/pre-commit <<'EOF'
npm --prefix ../ecom-debug-bot run extract
npm --prefix ../ecom-debug-bot run check-rules
git add .knowledge/generated
EOF
```

This assumes both repos sit side by side in `~/work`. Teammates without the tooling repo can skip the hook with `git commit --no-verify`, or you can move these two scripts into the backend repo later.

10.4 Nightly job on your machine

```
crontab -e
# add this line (runs at 02:00 every day):
0 2 * * * cd ~/work/ecom-debug-bot && /usr/bin/env bash -lc 'npm run nightly'
>> ~/work/ecom-debug-bot/nightly.log 2>&1
```

On macOS, give your terminal app Full Disk Access if cron can't read the repos.

10.5 CI guard (optional)

In your backend CI, add a step that fails a pull request when offer code changes but the rules file doesn't:

```
if git diff --name-only origin/main...HEAD | grep -q '^src/offers/' && \
   ! git diff --name-only origin/main...HEAD | grep -q '^.knowledge/rules/';
then
  echo "Offer logic changed: please review .knowledge/rules/offer-
eligibility.yaml"; exit 1
fi
```

Verify: rename a function referenced in the rules file and try to commit. The hook should print `STALE RULE` and block the commit.

Troubleshooting and progress checklist

Symptom	Likely cause	Fix
Model answers without calling tools	Model too small, or tools not enabled in chat	Use qwen2.5-coder 14B+; enable tools toggle (Open WebUI) or Agent mode (Continue); keep system prompt step 2
Model invents data	Tool error ignored	Check tool output in the Inspector; keep "never guess" rule in system prompt; lower temperature to 0
MCP server disconnects immediately	Something printed to stdout	Replace `console.log` with `console.error` everywhere the server imports, in the tooling repo (backend logs are safe inside the runner's child process)
`Cannot find module '@/...'` when the runner loads dist/	Backend path aliases	Copy `paths` + `baseUrl` only affects tooling files; for dist/, reuse whatever production uses to resolve aliases (tsc-alias or module-alias)
HTTP 401/403 from stage APIs	Wrong or expired token	Refresh the token in `.env.stage`; confirm it's a read-only service token

Symptom	Likely cause	Fix
`Trace stops early; call listed under BLOCKED WRITES`	A downstream POST that only reads (search APIs) was blocked	Add its path to allow_post_paths in environments.yaml (Phase 6.3)
Answers slow (> 60 s)	Context too large or model too big for RAM	Lower `num_ctx` to 16384; trim tool outputs; use 14B
Empty Jaeger	`OTEL_ENABLED` not set, or tracing not imported first	Check both; confirm port 4318 is mapped
Summaries are wrong	LLM misread code	Fix the `.md` by hand; the next nightly run only rewrites changed modules

Checklist

- [ ] Phase 1: Ollama running, `ecom-debugger` model passes the tool-call test
- [ ] Phase 2: Open WebUI or Continue chatting with the local model
- [ ] Phase 3: tooling repo created, dependencies installed
- [ ] Phase 4: `generated/` files produced, module notes reviewed
- [ ] Phase 5: INDEX, AGENTS, rules, environments, glossary written
- [ ] Phase 6: createServer + worker registry merged, runner CLI prints a trace
- [ ] Phase 7: `reproduce_request` and run_worker work in the Inspector, then in chat
- [ ] Phase 8: one job traced in Jaeger, outputs profiled
- [ ] Phase 9: `search_code` returns relevant functions
- [ ] Phase 10: pre-commit hook and nightly job running

Once the offer flow works end to end, extending to the next domain (pricing, promotions, cart, another worker) needs no new tools: add its folder to `WATCH`, register any new worker in the registry, and write its rules YAML and module notes so the bot can explain what the trace shows.